

## **DBMS Transactions-2**

focuses on learning how to work with data in a real database environment. It covers important SQL operations such as adding, updating, deleting, and viewing records from tables like Employees, Departments, Customers, Orders, and Products. The lab also helps in understanding how different tables are connected using joins, how to calculate values such as total employees and average salary using aggregate functions, and how to solve advanced problems with subqueries and window functions. Through these exercises, students gain practical knowledge of managing data, analyzing information,

and writing efficient SQL queries, which are essential skills for database development and management.

```
Your environment has been set up for using Node.js 24.16.0 (x64) and npm.

C:\Users\sriha>cd C:\Program Files\MySQL\MySQL Server 8.0\bin

C:\Program Files\MySQL\MySQL Server 8.0\bin>mysql -u root -p
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 19
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE college_db;
ERROR 1049 (42000): Unknown database 'college_db'
mysql> CREATE DATABASE college_db;
Query OK, 1 row affected (0.04 sec)

mysql> USE college_db;
Database changed
mysql> CREATE TABLE Departments (
    ->     dept_id INT PRIMARY KEY,
    ->     dept_name VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.11 sec)

mysql> CREATE TABLE Employees (
    ->     emp_id INT PRIMARY KEY,
    ->     emp_name VARCHAR(50),
    ->     salary INT,
    ->     dept_id INT,
    ->     manager_id INT,
    ->     email VARCHAR(100),
    ->     FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)
    -> );
```

```
-> );  
Query OK, 0 rows affected (0.06 sec)
```

```
mysql> INSERT INTO Departments VALUES
```

```
-> (1, 'CSE'),  
-> (2, 'ECE'),  
-> (3, 'MECH');
```

```
Query OK, 3 rows affected (0.02 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> INSERT INTO Employees VALUES
```

```
-> (101, 'Rahul', 70000, 1, NULL, 'rahul@gmail.com'),  
-> (102, 'Anil', 50000, 1, 101, 'anil@gmail.com'),  
-> (103, 'Priya', 65000, 2, 101, 'priya@gmail.com'),  
-> (104, 'Ravi', 25000, NULL, 101, 'ravi@gmail.com');
```

```
Query OK, 4 rows affected (0.01 sec)  
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM Employees;
```

| emp_id | emp_name | salary | dept_id | manager_id | email           |
|--------|----------|--------|---------|------------|-----------------|
| 101    | Rahul    | 70000  | 1       | NULL       | rahul@gmail.com |
| 102    | Anil     | 50000  | 1       | 101        | anil@gmail.com  |
| 103    | Priya    | 65000  | 2       | 101        | priya@gmail.com |
| 104    | Ravi     | 25000  | NULL    | 101        | ravi@gmail.com  |

```
4 rows in set (0.04 sec)
```

```
mysql> INSERT INTO Employees VALUES
```

```
-> (105, 'Sita', 60000, 1, 101, 'sita@gmail.com');
```

```
Query OK, 1 row affected (0.10 sec)
```

```
mysql> UPDATE Employees
```

```
-> SET salary = salary * 1.10  
-> WHERE dept_id = 1;
```

```
Query OK, 3 rows affected (0.03 sec)  
Rows matched: 3 Changed: 3 Warnings: 0
```

```
mysql> DELETE FROM Employees WHERE salary < 30000;
```

```
Query OK, 1 row affected (0.03 sec)
```

```
-> WHERE dept_id = 1;  
Query OK, 3 rows affected (0.03 sec)  
Rows matched: 3  Changed: 3  Warnings: 0
```

```
mysql> DELETE FROM Employees WHERE salary < 30000;  
Query OK, 1 row affected (0.03 sec)
```

```
mysql> SELECT e.emp_name, d.dept_name  
-> FROM Employees e  
-> JOIN Departments d  
-> ON e.dept_id = d.dept_id;
```

| emp_name | dept_name |
|----------|-----------|
| Rahul    | CSE       |
| Anil     | CSE       |
| Sita     | CSE       |
| Priya    | ECE       |

```
4 rows in set (0.04 sec)
```

```
mysql> SELECT e.emp_name  
-> FROM Employees e  
-> LEFT JOIN Departments d  
-> ON e.dept_id = d.dept_id  
-> WHERE d.dept_id IS NULL;  
Empty set (0.00 sec)
```

```
mysql> SELECT d.dept_name, e.emp_name  
-> FROM Departments d  
-> LEFT JOIN Employees e  
-> ON d.dept_id = e.dept_id;
```

| dept_name | emp_name |
|-----------|----------|
| CSE       | Rahul    |
| CSE       | Anil     |
| CSE       | Sita     |
| ECE       | Priya    |
| MECH      | NULL     |

```
+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> SELECT e.emp_name AS Employee,
->         m.emp_name AS Manager
-> FROM Employees e
-> LEFT JOIN Employees m
-> ON e.manager_id = m.emp_id;
```

```
+-----+-----+
| Employee | Manager |
+-----+-----+
| Rahul    | NULL    |
| Anil     | Rahul   |
| Priya    | Rahul   |
| Sita     | Rahul   |
+-----+-----+
```

```
4 rows in set (0.02 sec)
```

```
mysql> SELECT COUNT(*) AS Total_Employees FROM Employees;
```

```
+-----+
| Total_Employees |
+-----+
|                4 |
+-----+
```

```
1 row in set (0.01 sec)
```

```
mysql> SELECT dept_id, AVG(salary) AS Avg_Salary
-> FROM Employees
-> GROUP BY dept_id;
```

```
+-----+-----+
| dept_id | Avg_Salary |
+-----+-----+
|        1 | 66000.0000 |
|        2 | 65000.0000 |
+-----+-----+
```

```
2 rows in set (0.04 sec)
```

```
mysql> SELECT dept_id, AVG(salary) AS Avg_Salary
-> FROM Employees
-> GROUP BY dept_id
-> HAVING AVG(salary) > 60000;
```

```

-> HAVING AVG(salary) > 60000;
+-----+-----+
| dept_id | Avg_Salary |
+-----+-----+
|      1 | 66000.0000 |
|      2 | 65000.0000 |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT MAX(salary) AS Highest, MIN(salary) AS Lowest FROM Employees;
+-----+-----+
| Highest | Lowest |
+-----+-----+
|   77000 |   55000 |
+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT emp_name, salary
-> FROM Employees
-> WHERE salary > (SELECT AVG(salary) FROM Employees);
+-----+-----+
| emp_name | salary |
+-----+-----+
| Rahul    |   77000 |
| Sita     |   66000 |
+-----+-----+
2 rows in set (0.03 sec)

mysql> SELECT emp_name
-> FROM Employees
-> WHERE dept_id = (
->     SELECT dept_id
->     FROM Employees
->     WHERE emp_name = 'Rahul'
-> );
+-----+
| emp_name |
+-----+
| Rahul    |
| Anil     |
| Sita     |

```

```
| Sita      |
+-----+
3 rows in set (0.04 sec)
```

```
mysql> SELECT emp_name, salary
-> FROM Employees
-> WHERE salary = (SELECT MAX(salary) FROM Employees);
+-----+-----+
| emp_name | salary |
+-----+-----+
| Rahul    | 77000  |
+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> SELECT emp_name, salary
-> FROM (
->     SELECT emp_name, salary,
->           DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
->     FROM Employees
-> ) x
-> WHERE rnk = 2;
+-----+-----+
| emp_name | salary |
+-----+-----+
| Sita     | 66000  |
+-----+-----+
1 row in set (0.04 sec)
```

```
mysql> SELECT emp_name, salary
-> FROM Employees e
-> WHERE salary > (
->     SELECT AVG(salary)
->     FROM Employees
->     WHERE dept_id = e.dept_id
-> );
+-----+-----+
| emp_name | salary |
+-----+-----+
| Rahul    | 77000  |
+-----+-----+
1 row in set (0.03 sec)
```

```
+-----+-----+  
| emp_name | salary |  
+-----+-----+  
| Rahul    | 77000  |  
+-----+-----+  
1 row in set (0.03 sec)
```

```
mysql> SELECT e.emp_name  
-> FROM Employees e  
-> JOIN Employees m  
-> ON e.manager_id = m.emp_id  
-> WHERE e.salary > m.salary;  
Empty set (0.04 sec)
```

```
mysql> SELECT email, COUNT(*)  
-> FROM Employees  
-> GROUP BY email  
-> HAVING COUNT(*) > 1;  
Empty set (0.03 sec)
```

```
mysql>  
mysql>
```